// GESTIÓN DE CALIDAD DE SISTEMAS · TDD Y CI/CD

Pruebas unitarias propias vs. pruebas generadas por IA

Kata del Tres en Raya: comparación de alcance, calidad y capacidad de detectar errores



Autor: Robert0h2002

Repositorio: github.com/Robert0h2002/tdd-tres-en-raya

Octubre 2026

// CONTEXTO

La kata del Tres en Raya

Tres requerimientos, cada uno con sus pruebas (presentación de TDD – CI/CD)

R1 · Colocar pieza

- Fuera del eje X → excepción
- Fuera del eje Y → excepción
- Lugar ocupado → excepción

R2 · Turnos

- El primer turno es de **X**
- Después de X juega **+**
- Después de + juega **X**

R3 · Ganador

- Sin condición de victoria → no hay ganador
- Línea horizontal completa
- Línea vertical completa
- Línea diagonal completa

API implementada: `String jugar(int x, int y)` (coordenadas 1–3) y `char siguienteJugador()`. Resultados: `"No hay ganador"`, `"X es el ganador"`, `"+ es el ganador"`.

// CÓMO SE HIZO LA COMPARACIÓN

Metodología

Las dos suites se ejecutan contra la misma implementación, sin modificar ninguna

Pruebas propias

TDD: Red → Green → Refactor

- Un commit por requerimiento (prueba + código mínimo)
- Commit de refactor sin cambiar comportamiento
- Java 17 + JUnit 5 + Maven + JaCoCo
- CI con GitHub Actions en cada push / PR

Pruebas de IA

Generadas a partir de un prompt

- El prompt incluye los 3 requerimientos y la API pública
- La IA no vio el código fuente ni mis pruebas
- El resultado se copió tal cual a `TresEnRayaIATest.java`
- Se ejecuta en el mismo pipeline de CI

Además de contar pruebas y cobertura, se aplicaron **10 mutantes** (errores introducidos a propósito en el código) para medir qué suite detecta cada error.

// ENTREGABLE

El prompt usado con la IA

Texto completo en `docs/prompt-ia.md` del repositorio

```
Actúa como un desarrollador Java con experiencia en TDD (Test Driven Development).

Necesito que escribas SOLO las pruebas unitarias (todavía no el código) para la kata
"Tres en Raya" (tic-tac-toe). Usa Java 17 y JUnit 5 (org.junit.jupiter).

Requerimientos de la kata:  [R1, R2 y R3 copiados de la presentación, con sus pruebas]

La clase a probar se llama TresEnRaya, está en el paquete com.ucb.tdd y tiene esta API pública:
  public String jugar(int x, int y);    // x, y de 1 a 3; RuntimeException si la jugada no es válida;
                                        // retorna "No hay ganador", "X es el ganador" o "+ es el ganador"
  public char siguienteJugador();       // 'X' o '+'

Condiciones:
  - Una clase de prueba llamada TresEnRayaIATest en el paquete com.ucb.tdd.
  - Nombres de métodos de prueba descriptivos en español.
  - Sigue el patrón Arrange-Act-Assert.
  - Cubre todos los escenarios de los requerimientos y agrega los casos borde
    que consideres importantes.
  - Devuelve solo el código Java de la clase de prueba, sin explicación.
```

// RESULTADOS

Comparación en números

Las dos suites pasan al 100 % y cubren el 100 % del código, pero no detectan los mismos errores

Casos de prueba	Líneas de código de prueba	Cobertura (líneas / ramas)	Mutantes detectados
13 propias	**118**	**100 %**	**8** de 10
36 IA	**448**	**100 %**	**7** de 10

ASPECTO	PROPIAS	IA
Verifica el mensaje de la excepción	**Sí**	**No**
Prueba las 3 filas y las 3 columnas	**No (solo 1 de cada una)**	**Sí**
Prueba que el turno no avance tras una jugada inválida	**No**	**Sí**
Tablero lleno sin ganador / victoria en la última casilla	**No**	**Sí**

// DIFERENCIA SUSTANCIAL 1

Cómo se verifican las excepciones

La IA solo verifica el tipo; mis pruebas también verifican el mensaje

Propia

```
@Test
void cuandoPiezaFueraDelEjeXEntoncesLanzaExcepcion() {
    RuntimeException e = assertThrows(
        RuntimeException.class, () -> juego.jugar(4, 2));
    assertEquals("X esta fuera del tablero",
                 e.getMessage());
}
```

IA

```
@Test
void cuandoLaPiezaEstaFueraDelEjeXPorEncima...() {
    // Arrange
    int x = 4;
    int y = 1;
    // Act + Assert
    assertThrows(RuntimeException.class,
                 () -> juego.jugar(x, y));
}
```

Consecuencia: si la validación del límite está mal (mutante M1: `x > 4`), Java lanza `ArrayIndexOutOfBoundsException`, que *también* es una `RuntimeException`. Las 36 pruebas de la IA siguen en verde; 2 de mis pruebas fallan y detectan el error.

// DIFERENCIA SUSTANCIAL 2

Amplitud de las líneas ganadoras

Yo escribí una prueba por tipo de línea; la IA probó cada fila, cada columna y cada jugador

Propias · 4 pruebas de victoria

- Horizontal: solo la fila `y = 1`
- Vertical: solo la columna `x = 1`
- Diagonal principal y diagonal secundaria
- Alterna ganador X y ganador +

Suficiente para hacer pasar el código en TDD (código mínimo), pero deja huecos.

IA · 12 pruebas de victoria + 6 de "no hay ganador"

- Las 3 filas y las 3 columnas para X
- Una fila, una columna y una diagonal para +
- Línea con fichas mezcladas, dos en línea sin la tercera
- Tablero lleno sin ganador y victoria en la última casilla

Consecuencia: si el ciclo solo revisa la primera fila/columna (mutante M6), **mis 13 pruebas pasan** y 5 pruebas de la IA fallan. Es el error que mi suite no detecta.

// DIFERENCIA SUSTANCIAL 3

Casos borde y estado del juego

La IA propuso escenarios que no estaban en los requerimientos

Solo en la IA

- Coordenadas negativas (`-1`, `-5`) y ambas fuera del tablero
- Lugar ocupado por el mismo jugador vs. por el oponente
- **El turno no avanza** después de una jugada inválida
- Consultar `siguienteJugador()` varias veces no cambia el turno
- Las 4 esquinas son posiciones válidas

Solo en las propias

- Mensaje exacto de cada excepción (eje X, eje Y, lugar ocupado)
- Prueba de límites agrupada (coordenada 0 en X y en Y)

Mis pruebas siguen exactamente la lista de la kata: más fieles al enunciado, pero menos exploratorias.

Las pruebas de estado de la IA son las más fuertes: el mutante M8 (el turno avanza aunque la jugada sea inválida) hace fallar **16 de 36** pruebas de la IA frente a 3 de 13 propias.

// DIFERENCIA SUSTANCIAL 4

Estilo, legibilidad y mantenimiento

Más pruebas no siempre significa mejores pruebas

ASPECTO	PROPIAS	IA
Estructura	Agrupadas por requerimiento, sin comentarios AAA	Comentarios `// Arrange / Act / Assert` explícitos
Nombres	Cortos, siguen el texto de la kata	Muy largos (hasta 70 caracteres)
Datos esperados	Literales en cada prueba	Constantes (`GANA_X`, `SIN_GANADOR`…)
Redundancia	Baja	Media: 3 pruebas casi iguales de "lugar ocupado" y 6 de "no hay ganador"
Una prueba = una idea	Sí (excepto la prueba de límites con 2 asserts)	No siempre: la prueba de esquinas hace 4 jugadas con 4 asserts
Ambigüedad	—	"Fila inferior/superior" asume una orientación del tablero que la kata no define
Proceso	Incremental, guiaron el diseño (TDD real)	Escritas de una sola vez después de conocer la API

// EVIDENCIA

Prueba de mutantes: ¿quién detecta cada error?

Se introdujo un error a la vez en `TresEnRaya.java` y se ejecutaron ambas suites (pruebas que fallan / total)

MUTANTE (ERROR INTRODUCIDO)	PROPIAS	IA
M1 · Límite superior mal (`> 4`)	**2/13 detecta**	**0/36 no detecta**
M2 · Límite inferior mal (`< 0`)	**0/13 no detecta**	**0/36 no detecta**
M3 · No valida lugar ocupado	**1/13**	**4/36**
M4 · Mensaje de excepción cambiado	**2/13**	**0/36 no detecta**
M5 · Sin diagonal secundaria	**1/13**	**2/36**
M6 · Solo revisa la primera fila/columna	**0/13 no detecta**	**5/36**
M7 · Sin líneas verticales	**1/13**	**4/36**
M8 · El turno avanza aunque la jugada sea inválida	**3/13**	**16/36**
M9 · Mensaje de ganador fijo en "X"	**2/13**	**3/36**
M10 · Mensaje "sin ganador" cambiado	**1/13**	**6/36**
Total detectados	8 / 10	7 / 10

// ANÁLISIS

Fortalezas y debilidades

Cada suite cubre los puntos ciegos de la otra

Propias

Fortalezas

- Pocas pruebas, fáciles de leer y mantener
- Verifican el contrato completo (tipo + mensaje)
- Nacieron con el código: guiaron el diseño

Debilidades

- Solo una fila y una columna probadas
- No prueban el estado después de un error

IA

Fortalezas

- Muy exhaustivas en combinaciones de victoria
- Proponen casos borde y de estado nuevos
- Generadas en segundos

Debilidades

- `assertThrows(RuntimeException)` demasiado genérico
- Redundantes y verbosas (448 líneas)
- Dependen de que el prompt describa bien la API

// CIERRE

Conclusiones

- **100 % de cobertura no garantiza buenas pruebas:** ambas suites cubren todo el código y aun así cada una deja pasar errores distintos.
- **La IA gana en amplitud** (más escenarios y casos borde) y **las pruebas propias en precisión** (verifican el mensaje y el contrato exacto).
- Ninguna suite detectó el mutante M2: probar con `assertThrows(RuntimeException.class)` sin revisar el mensaje oculta errores de índice. Conviene usar una excepción propia o verificar el mensaje.
- La IA no reemplaza el ciclo TDD: sus pruebas se escriben después de conocer la API, no guían el diseño.
- **Lo más útil es combinarlas:** escribir las pruebas con TDD y usar la IA como revisor para encontrar casos que faltan.

> Repositorio: **github.com/Robert0h2002/tdd-tres-en-raya** · Prompt: `docs/prompt-ia.md` · CI: GitHub Actions (`mvn clean test`)